

Week 1 — Assignment Problems — Solutions

Week 1 Assignment — 5 Problems

Name: Gursimran Bajwa | **Registration No:** RA2511003011289**Language:** Java | **Problems solved:** 5 / 5

Problem 1 — The Exam Hall Seat Duplication Checker

Problem Statement

The Examination Cell must confirm that no seat number has been assigned to two different students before an exam begins. The tool scans the full list of assigned seat numbers and flags any duplicates. Constraint: arrays and loops only, no Collections class.

Approach

- A nested loop compares each seat number with every later one (j starts at $i + 1$), which avoids comparing a pair twice.
- A second small int array, `alreadyReported`, records the duplicates that have already been printed so a value repeated three or more times is still reported only once.
- A boolean flag tracks whether any duplicate was found at all; if it is still false after both loops, the confirmation message is printed.
- No Collections class, HashSet or Arrays utility is used — only plain arrays, loops and conditionals, as required.

Java Program — ExamHallSeatDuplicationChecker.java

```
/**
 * Problem 1 - The Exam Hall Seat Duplication Checker
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Constraint: arrays and loops only - no Collections classes used.
 */
public class ExamHallSeatDuplicationChecker {

    /* Compares every seat number with every later one and reports duplicates. */
    public static void checkDuplicateSeats(int[] seatNumbers) {
        boolean duplicateFound = false;
        int[] alreadyReported = new int[seatNumbers.length];
        int reportedCount = 0;

        for (int i = 0; i < seatNumbers.length; i++) {
            for (int j = i + 1; j < seatNumbers.length; j++) {
                if (seatNumbers[i] == seatNumbers[j]) {

                    // Make sure the same duplicate is not printed twice.
                    boolean alreadyPrinted = false;
                    for (int k = 0; k < reportedCount; k++) {
                        if (alreadyReported[k] == seatNumbers[i]) {
                            alreadyPrinted = true;
                            break;
                        }
                    }

                    if (!alreadyPrinted) {
                        System.out.println("Duplicate Seat Number Found: " + seatNumbers[i]);
                        alreadyReported[reportedCount] = seatNumbers[i];
                        reportedCount++;
                        duplicateFound = true;
                    }
                }
            }
        }
    }
}
```

```
        if (!duplicateFound) {
            System.out.println("No Duplicate Seats Found");
        }
    }

    private static void printArray(int[] array) {
        StringBuilder line = new StringBuilder("{");
        for (int i = 0; i < array.length; i++) {
            line.append(array[i]);
            if (i < array.length - 1) line.append(", ");
        }
        System.out.println("Seat list: " + line.append("}"));
    }

    public static void main(String[] args) {
        int[] hallOne = { 101, 102, 103, 102, 105 };
        int[] hallTwo = { 101, 102, 103, 104, 105 };
        int[] hallThree = { 201, 202, 201, 204, 202, 206 };

        printArray(hallOne);
        checkDuplicateSeats(hallOne);
        System.out.println();

        printArray(hallTwo);
        checkDuplicateSeats(hallTwo);
        System.out.println();

        printArray(hallThree);
        checkDuplicateSeats(hallThree);
    }
}
```

Output

```
Seat list: {101, 102, 103, 102, 105}
Duplicate Seat Number Found: 102

Seat list: {101, 102, 103, 104, 105}
No Duplicate Seats Found

Seat list: {201, 202, 201, 204, 202, 206}
Duplicate Seat Number Found: 201
Duplicate Seat Number Found: 202
```

Problem 2 — The Typing Speed Test Accuracy Checker

Problem Statement

An online typing-practice site compares a user's attempt against the original passage character by character, and reports the accuracy percentage along with the position of the very first mistake.

Approach

- Lengths are validated first, since the comparison is position-by-position and assumes equal-length strings.
- A single loop uses `charAt(i)` on both strings: matching positions increment the counter, and the first mismatching index is stored once (guarded by a -1 check) so later mismatches do not overwrite it.
- Accuracy is $(\text{matched} * 100.0) / \text{total}$, printed to two decimals with `String.format()`.
- The reported position is `index + 1` so it reads as a 1-based position for the user, matching the expected output.

Java Program — `TypingSpeedAccuracyChecker.java`

```
/**
 * Problem 2 - The Typing Speed Test Accuracy Checker
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class TypingSpeedAccuracyChecker {

    /* Compares the typed text with the original, position by position. */
    public static void checkTypingAccuracy(String original, String typed) {
        if (original.length() != typed.length()) {
            System.out.println("Error: both strings must be of equal length.");
            return;
        }

        int total = original.length();
        int matched = 0;
        int firstMismatchPosition = -1;

        for (int i = 0; i < total; i++) {
            if (original.charAt(i) == typed.charAt(i)) {
                matched++;
            } else if (firstMismatchPosition == -1) {
                firstMismatchPosition = i; // remember the very first slip
            }
        }

        double accuracy = (matched * 100.0) / total;
        System.out.print("Matched: " + matched + "/" + total
            + " | Accuracy: " + String.format("%.2f", accuracy) + "%");

        if (firstMismatchPosition == -1) {
            System.out.println(" | No Mismatches");
        } else {
            System.out.println(" | First Mismatch at position " + (firstMismatchPosition + 1)
                + " ('" + original.charAt(firstMismatchPosition)
                + "' vs '" + typed.charAt(firstMismatchPosition) + "')");
        }
    }

    public static void main(String[] args) {
        String[][] attempts = {
            { "hello world", "hello worlt" },
            { "coding", "coding" },
            { "java strings", "jawa strimgs" }
        };

        for (String[] attempt : attempts) {
            System.out.println("Original: \"" + attempt[0] + "\" | Typed: \"" + attempt[1] + "\"");
            checkTypingAccuracy(attempt[0], attempt[1]);
            System.out.println();
        }
    }
}
```

Output

Original: "hello world" | Typed: "hello worlt"
Matched: 10/11 | Accuracy: 90.91% | First Mismatch at position 11 ('d' vs 't')

Original: "coding" | Typed: "coding"
Matched: 6/6 | Accuracy: 100.00% | No Mismatches

Original: "java strings" | Typed: "jawa strimngs"
Matched: 10/12 | Accuracy: 83.33% | First Mismatch at position 3 ('v' vs 'w')

Problem 3 — The Traffic Signal Streak Analyzer

Problem Statement

The traffic control department logs a signal's colour every minute as 'R', 'Y' or 'G'. Engineers need the longest continuous streak of the same colour so they know which signal is getting stuck.

Approach

- One linear pass, tracking the current streak's colour and length alongside the best streak seen so far — $O(n)$ time and $O(1)$ space.
- When the next character equals the current colour the streak length grows; otherwise a new streak of length 1 starts at that character.
- After every step the current streak is compared against the best; using a strict $>$ keeps the earliest streak when two are tied in length.
- Both trackers are initialised from character 0, and an empty-log guard prevents an index error.

Java Program — TrafficSignalStreakAnalyzer.java

```
/**
 * Problem 3 - The Traffic Signal Streak Analyzer
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class TrafficSignalStreakAnalyzer {

    /* Single pass over the log, tracking the current streak and the best one so far. */
    public static void findLongestStreak(String signalLog) {
        if (signalLog == null || signalLog.isEmpty()) {
            System.out.println("Empty signal log - nothing to analyze.");
            return;
        }

        char bestColor = signalLog.charAt(0);
        int bestLength = 1;

        char currentColor = signalLog.charAt(0);
        int currentLength = 1;

        for (int i = 1; i < signalLog.length(); i++) {
            if (signalLog.charAt(i) == currentColor) {
                currentLength++; // streak continues
            } else {
                currentColor = signalLog.charAt(i); // streak broken, start a new one
                currentLength = 1;
            }

            if (currentLength > bestLength) {
                bestLength = currentLength;
                bestColor = currentColor;
            }
        }

        System.out.println("Longest Streak: '" + bestColor + "' repeated " + bestLength + " times");
    }

    public static void main(String[] args) {
        String[] logs = { "RRGGGYRR", "RRRRYYGG", "RYGRYGRYG", "GGGGGGRY" };

        for (String log : logs) {
            System.out.println("Signal log: \"" + log + "\"");
            findLongestStreak(log);
            System.out.println();
        }
    }
}
```

Output

```
Signal log: "RRGGGYRR"
Longest Streak: 'G' repeated 3 times
```

Signal log: "RRRRYYGG"
Longest Streak: 'R' repeated 4 times

Signal log: "RYGRYGRYG"
Longest Streak: 'R' repeated 1 times

Signal log: "GGGGGGRY"
Longest Streak: 'G' repeated 6 times

Problem 4 — The Warehouse Inventory Balancer

Problem Statement

A warehouse stores the same product categories in Section A and Section B. Before the monthly stock report, the team must confirm both sections hold matching totals and identify the single highest-quantity item across the whole warehouse.

Approach

- Because both arrays are the same length, a single loop accumulates both totals at once.
- The totals are compared directly to print "Balanced" or "Not Balanced".
- The maximum is tracked together with the section name and the index where it occurred: Section A is scanned first, then Section B, using a strict $>$ so the earliest occurrence wins a tie (this is why 30 is reported at Section A, Item 3 in the sample).
- The reported item number is index + 1 so it reads as a 1-based item position.

Java Program — WarehouseInventoryBalancer.java

```
/**
 * Problem 4 - The Warehouse Inventory Balancer
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class WarehouseInventoryBalancer {

    /* Totals both sections, compares them, and locates the single highest quantity. */
    public static void analyzeInventory(int[] sectionA, int[] sectionB) {
        if (sectionA.length != sectionB.length) {
            System.out.println("Error: both sections must have the same number of items.");
            return;
        }

        int totalA = 0, totalB = 0;
        for (int i = 0; i < sectionA.length; i++) {
            totalA += sectionA[i];
            totalB += sectionB[i];
        }

        String status = (totalA == totalB) ? "Balanced" : "Not Balanced";

        // Track the maximum together with the section and index where it was found.
        int highest = sectionA[0];
        String highestSection = "Section A";
        int highestIndex = 0;

        for (int i = 1; i < sectionA.length; i++) {
            if (sectionA[i] > highest) {
                highest = sectionA[i];
                highestSection = "Section A";
                highestIndex = i;
            }
        }
        for (int i = 0; i < sectionB.length; i++) {
            if (sectionB[i] > highest) {
                highest = sectionB[i];
                highestSection = "Section B";
                highestIndex = i;
            }
        }

        System.out.println("Section A Total: " + totalA
            + " | Section B Total: " + totalB
            + " | Status: " + status
            + " | Highest Quantity: " + highest
            + " (" + highestSection + ", Item " + (highestIndex + 1) + ")");
    }

    public static void main(String[] args) {
        int[] a1 = { 20, 15, 30 };
        int[] b1 = { 25, 10, 30 };
        analyzeInventory(a1, b1);
    }
}
```

```
int[] a2 = { 40, 12, 8, 5 };
int[] b2 = { 10, 60, 9, 7 };
analyzeInventory(a2, b2);

int[] a3 = { 5, 5, 5 };
int[] b3 = { 5, 5, 5 };
analyzeInventory(a3, b3);
}
```

Output

```
Section A Total: 65 | Section B Total: 65 | Status: Balanced | Highest Quantity: 30 (Section A, Item 3)
Section A Total: 65 | Section B Total: 86 | Status: Not Balanced | Highest Quantity: 60 (Section B, Item 2)
Section A Total: 15 | Section B Total: 15 | Status: Balanced | Highest Quantity: 5 (Section A, Item 1)
```


Problem 5 — The Movie Review Word Length Profiler

Problem Statement

A moderation tool profiles the word lengths in a newly submitted review, since reviews stuffed with unusually many very short or very long words are more likely to be spam.

Approach

- The review is trimmed and split on the regular expression `\s+`, so runs of multiple spaces do not create empty words.
- Each word's `length()` drives a three-way if / else-if chain: 1–4 Short, 5–8 Medium, 9 or more Long.
- Three counters accumulate the buckets and are printed in one formatted line.
- Extra reviews (all-short and all-long) are included to show every branch of the classification firing.

Java Program — `MovieReviewWordLengthProfiler.java`

```
/**
 * Problem 5 - The Movie Review Word Length Profiler
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class MovieReviewWordLengthProfiler {

    /* Splits the review and buckets every word by its length. */
    public static void classifyWordLengths(String review) {
        String[] words = review.trim().split("\\s+");

        int shortWords = 0, mediumWords = 0, longWords = 0;

        for (String word : words) {
            if (word.isEmpty()) continue;

            int length = word.length();
            if (length <= 4) shortWords++; // Short : 1-4 letters
            else if (length <= 8) mediumWords++; // Medium : 5-8 letters
            else longWords++; // Long : 9+ letters
        }

        System.out.println("Short: " + shortWords
            + " | Medium: " + mediumWords
            + " | Long: " + longWords);
    }

    public static void main(String[] args) {
        String[] reviews = {
            "This movie was absolutely fantastic and thrilling",
            "Bad plot slow pace",
            "An extraordinarily breathtaking cinematographic achievement"
        };

        for (String review : reviews) {
            System.out.println("Review: \"" + review + "\"");
            classifyWordLengths(review);
            System.out.println();
        }
    }
}
```

Output

```
Review: "This movie was absolutely fantastic and thrilling"
Short: 3 | Medium: 1 | Long: 3

Review: "Bad plot slow pace"
Short: 4 | Medium: 0 | Long: 0

Review: "An extraordinarily breathtaking cinematographic achievement"
Short: 1 | Medium: 0 | Long: 4
```